مصفوفة التكرار

Frequency Array & Count Sort

كيف نحوّل مصفوفة أرقام إلى جدول تكرار؟ وكيف نستخدمه في الترتيب؟

مصفوفات 1D | مصفوفة التكرار | Count Sort

إعداد: الأستاذ محمود اغبارية

الفكرة الأساسية

عندنا مصفوفة تحتوي أرقامًا بين **0** و**9**. نريد أن نعرف: **كم مرة ظهر كل رقم؟**

الفكرة الذكية: نُنشئ مصفوفة طولها **10** دائمًا، **رقم القيمة = فهرس الخانة.**

المصفوفة الأصلية

0	1	2	3	4	5	6
5	2	5	8	2	5	0

→

مصفوفة التكرار (طولها 10 دائمًا)

0	1	2	3	4	5	6	7	8	9
1	0	2	0	0	3	0	0	1	0

السطر الجوهري: `frequency[arr[i]]++` – قيمة العنصر تُستخدم كـ **فهرس** في مصفوفة التكرار.

CreateFrequencyArray الكود – </>

```
public static int[] CreateFrequencyArray(int[] arr)
{
    int[] frequency = new int[10];

    for (int i = 0; i < arr.Length; i++)
    {
        frequency[arr[i]]++;   // القيمة = الفهرس
    }

    return frequency;
}
```

السطر المهم:

`frequency[arr[i]]++`

arr[i] هي القيمة الحالية (مثلاً 5).

نذهب إلى الخانة رقم **5** في `frequency` ونزيدها.

ملاحظة: المصفوفة `frequency` تُهيَّأ تلقائيًا بأصفار في C#.

تتبع خطوة بخطوة – CreateFrequencyArray

نُنشئ مصفوفة frequency طولها 10، كل قيمة تبدأ بـ 0.

i = - arr[i] = - frequency[arr[i]] = -

arr = {5, 2, 5, 8, 2, 5, 0}

0	1	2	3	4	5	6
5	2	5	8	2	5	0

frequency (طولها 10)

0	1	2	3	4	5	6	7	8	9
0	0	0	0	0	0	0	0	0	0

◀ السابق 1 / 9 التالي ▶

تمرين ١ – الأكثر تكرارًا

اكتب عملية خارجية `int FindMostFrequent(int[] arr)` تتلقى مصفوفة أعداد صحيحة (تحتوي فقط على الأرقام من 0 إلى 9) وتعيد الرقم الأكثر تكرارًا فيها.

مثال:

```
arr = {3, 1, 3, 7, 1, 3, 7, 7, 3} // الناتج: 3 (ظهر 4 مرات)
```

إذا تساوى عددان في التكرار، أعد الأصغر منهما.

تلميح:

- ابنِ أولًا مصفوفة التكرار.
- ابحث عن الخانة التي فيها أكبر قيمة.
- فهرس تلك الخانة هو الناتج.

✅ حل تمرين ١ – FindMostFrequent

✓ الحل

```
public static int FindMostFrequent(int[] arr)
{
    int[] frequency = CreateFrequencyArray(arr);

    int maxIdx = 0;
    for (int i = 1; i < frequency.Length; i++)
    {
        if (frequency[i] > frequency[maxIdx])  // > وليس >= للحصول على الأصغر عند التساوي
            maxIdx = i;
    }

    return maxIdx;
}
```

ملاحظة: نبدأ من `i = 1` ونقارن بـ `frequency[maxIdx]` وليس بقيمة عددية – هذا النمط مفيد دائمًا عند البحث عن الأكبر في مصفوفة.

تمرين ٢ – هل كل رقم ظهر مرة واحدة فقط؟

اكتب عملية خارجية `bool AllUnique(int[] arr)` تتلقى مصفوفة أعداد صحيحة (من 0 إلى 9) وتعيد **true** إذا كانت جميع الأرقام الموجودة لا تتكرر (كل رقم يظهر مرة واحدة على الأكثر)، وإلا تعيد **false**.

أمثلة:

`{3, 1, 7, 5, 9}` → true `{3, 1, 7, 3, 9}` → false // الرقم 3 تكرّر

تحدٍّ للتفكير: كيف تحل المسألة **بدون** مصفوفة تكرار؟ وأي الحلين أسرع؟

حل تمرين ٢ – AllUnique

الحل

```
public static bool AllUnique(int[] arr)
{
    int[] frequency = CreateFrequencyArray(arr);

    for (int i = 0; i < frequency.Length; i++)
    {
        if (frequency[i] > 1)
            return false;
    }

    return true;
}
```

لماذا هذا أفضل من حلقة مضاعفة؟ الحل بمصفوفة التكرار يمر على المصفوفة **مرتين فقط**، بينما الحلقة المضاعفة تمر **n × n** مرة.

تمرين ٣ – كم رقمًا مختلفًا؟

اكتب عملية خارجية `int CountDistinct(int[] arr)` تتلقى مصفوفة أعداد صحيحة (من 0 إلى 9) وتعيد عدد الأرقام المختلفة (المتمايزة) الموجودة فيها.

مثال:

`{5, 2, 5, 8, 2, 5, 0}` → `4` // الأرقام المختلفة: 0, 2, 5, 8

تلميح: بعد بناء مصفوفة التكرار، عدّ كم خانة قيمتها **أكبر من صفر**.

✓ الحل

```
public static int CountDistinct(int[] arr)
{
    int[] frequency = CreateFrequencyArray(arr);

    int count = 0;
    for (int i = 0; i < frequency.Length; i++)
    {
        if (frequency[i] > 0)
            count++;
    }

    return count;
}
```

النمط المشترك في كل التمارين:

١. ابنِ مصفوفة التكرار.

٢. عالج مصفوفة التكرار للإجابة على السؤال.

من التكرار إلى الترتيب

سؤال للتفكير: إذا كانت عندنا مصفوفة التكرار، هل يمكننا ترتيب المصفوفة الأصلية **دون أن نقارن عنصرين ببعض؟**

مصفوفة التكرار لـ `{3, 1, 3, 2, 1, 3}`

0	1	2	3	4	5	6	7	8	9
0	2	1	3	0	0	0	0	0	0

مصفوفة التكرار تقول:

الرقم **1** ظهر **مرتين** ← اكتب 1، 1

الرقم **2** ظهر **مرة** ← اكتب 2

الرقم **3** ظهر **ثلاثًا** ← اكتب 3، 3، 3

النتيجة: `{1, 1, 2, 3, 3, 3}` – مرتّبة تصاعديًا!

Count Sort – الكود

```
public static void CountSort(int[] arr)
{
    int[] frequency = CreateFrequencyArray(arr);

    int idx = 0;
    for (int digit = 0; digit ≤ 9; digit++)
    {
        for (int j = 0; j < frequency[digit]; j++)
        {
            arr[idx] = digit;
            idx++;
        }
    }
}
```

`frequency[digit]`

كم مرة نكتب الرقم `digit` في المصفوفة؟

الخطوات:

1. ابنِ مصفوفة التكرار.
2. لكل رقم من 0 إلى 9، اكتبه بقدر عدد تكراره.
3. المصفوفة الأصلية مرتّبة الآن.

محاكاة Count Sort

نُنشئ مصفوفة التكرار من arr.

digit = - idx = - frequency[digit] = -

arr (يتغير أثناء الترتيب)

0	1	2	3	4	5	6	7
3	1	3	2	1	3	0	2

frequency

0	1	2	3	4	5	6	7	8	9
0	0	0	0	0	0	0	0	0	0

◀ السابق 1 / 18 التالي ▶

⚖️ Count Sort مقارنةً بـ Insertion Sort

الخاصية	Insertion Sort	Count Sort
طريقة العمل	مقارنة ونقل عناصر	عدّ ثم إعادة كتابة
القيود	يعمل مع أي أعداد	يحتاج نطاقًا معروفًا ومحدودًا
السرعة	بطيء مع مصفوفات كبيرة	سريع جدًا عندما النطاق صغير
الذاكرة الإضافية	لا يحتاج	يحتاج مصفوفة التكرار

Count Sort مثالي عندما: القيم محدودة ومعروفة مسبقًا (مثل 0-9، أو درجات 0-100).

لا يصلح إذا: كانت القيم كبيرة جدًا (بالملايين)، لأن مصفوفة التكرار ستكون ضخمة.

ملخص الدرس

مصفوفة التكرار

- طولها ثابت (10 في مسائلنا)
- الفهرس = قيمة الرقم
- المحتوى = عدد مرات الظهور
- السطر الجوهري: `frequency[arr[i]]++`

استخدامات مصفوفة التكرار

- الرقم الأكثر تكرارًا
- التحقق من التفرد
- عدد الأرقام المتمايزة
- الترتيب (Count Sort)

Count Sort – الخطوات

1. ابنِ مصفوفة التكرار.
2. لكل رقم، اكتبه بقدر تكراره.
3. المصفوفة مرتّبة تصاعديًا.

النمط العام:

١. ابنِ مصفوفة التكرار.

٢. اعمل عليها لحل المسألة.

إعداد: الأستاذ محمود اغبارية